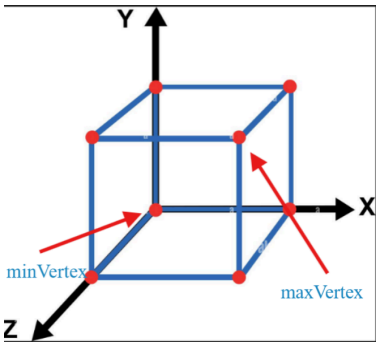
Nama : Wildan Abdurrahman Ghazali
NIM: 13524054

Laporan Tugas Kecil 2 IF2211 Strategi Algoritma : Voxelization Objek 3D menggunakan Octree

Program Utama

A. Struktur data Utama:

1. Vertex yang menyimpan koordinat x,y,z dari input file .obj
2. Face yang menyimpan segitiga yang terdiri dari 3 sudut vertex
3. Octree sebagai pohon utama, berisi :
   a. OctreeNode root
   b. Kedalaman maksimum
   c. Kumpulan vertex dan face awal
   d. Kumpulan vertex dan face setelah di voxelisasi
4. OctreeNode sebagai Node tiap voxel, menyimpan
   a. vertex minimum dan maximum yang berfungsi sebagai bounding box.
   b. 8 OctreeNode children
   c. Kedalaman pohon
   d. Kumpulan sisi yang berpotongan



Gambar kubus dengan minVertex dan maxVertex sebagai bounding box

B. Algoritma

Voxel / kubus direpresentasikan sebagai 8 vertex (titik sudut kubus) dan 12 face (sisi) segitiga (6 sisi, tiap sisi dibangun dari 2 segitiga). Sebelum melakukan divide and conquer, praproses data dari file .obj:

1. Input data vertices dan faces dari file .obj ke vector, serta kedalaman maksimum (maxDepth).
2. Inisialisasi Octree

```
Vertex minV = minVertex(vertices);
Vertex maxV = maxVertex(vertices);
Octree octree(minV, maxV, maxDepth, vertices, faces);
```

```
Octree::Octree(Vertex min, Vertex max, int maxDepth,
            const std::vector<Vertex>& vertices,
            const std::vector<Face>& faces)
{
    root = createOctreeNode(min, max, 0);
    this->maxDepth = maxDepth;
    this->initialVertices = vertices;
    this->initialFaces = faces;
    for (int i = 0; i < (int)faces.size(); i++)
        root->intersectFaces.push_back(i);
}
```

Selanjutnya, lakukan Divide and Conquer

1. Divide

   Tahap ini direpresentasikan oleh prosedur

```
void Octree::divide(OctreeNode* node)
```

   Setiap node (kubus) dibagi menjadi 8. Langkah-langkah:

   a. Hitung titik tengah bounding box:

```
Vertex mid{
```

```
        (node->min.x + node->max.x) / 2,
        (node->min.y + node->max.y) / 2,
        (node->min.z + node->max.z) / 2
    };
```

b. Bagi menjadi 8 child node. Setiap child memiliki kombinasi min/max berdasarkan titik tengah

```
for (int i = 0; i < 8; i++) {
        Vertex minChild;
        Vertex maxChild;

        switch (i)
        {
            case 0:
                minChild = Vertex{node->min.x, node->min.y, node->min.z};
                maxChild = Vertex{mid.x, mid.y, mid.z};
                break;

            case 1:
                minChild = Vertex{mid.x, node->min.y, node->min.z};
                maxChild = Vertex{node->max.x, mid.y, mid.z};
                break;

            case 2:
                minChild = Vertex{node->min.x, mid.y, node->min.z};
                maxChild = Vertex{mid.x, node->max.y, mid.z};
                break;

            case 3:
                minChild = Vertex{mid.x, mid.y, node->min.z};
                maxChild = Vertex{node->max.x, node->max.y, mid.z};
                break;

            case 4:
                minChild = Vertex{node->min.x, node->min.y, mid.z};
                maxChild = Vertex{mid.x, mid.y, node->max.z};
                break;

            case 5:
                minChild = Vertex{mid.x, node->min.y, mid.z};
                maxChild = Vertex{node->max.x, mid.y, node->max.z};
                break;

            case 6:
```

```
                minChild = Vertex{node->min.x, mid.y, mid.z};
                maxChild = Vertex{mid.x, node->max.y, node->max.z};
                break;

            case 7:
                minChild = Vertex{mid.x, mid.y, mid.z};
                maxChild = Vertex{node->max.x, node->max.y, node->max.z};
                break;
        }
         node->children[i] = createOctreeNode(minChild, maxChild,
node->depth + 1);
```

c. Rekursi:

```
divide(node->children[i]);
```

Base case:

i. Sudah mencapai kedalaman maksimum, atau
ii. Tidak ada face yang relevan di node tersebut

```
if (node->depth >= maxDepth || node->intersectFaces.empty()) {

        node->isLeaf = true;

        return;

    }
```

2. Conquer

Pada setiap children, Filter face yang beririsan dengan node children berdasarkan bounding box children dari kumpulan instersectFaces parent:

```
for (int idx : node->intersectFaces) {

     if (intersect(node->children[i], initialFaces[idx]))
node->children[i]->intersectFaces.push_back(idx);

}
```

Fungsi intersect() bertujuan untuk menentukan apakah sebuah segitiga beririsan dengan sebuah voxel pada node octree. Metode yang digunakan adalah Separating Axis Theorem (SAT), yang merupakan metode collision detection antara dua objek.

SAT menyatakan bahwa dua objek konveks tidak beririsan jika terdapat suatu sumbu di mana proyeksi kedua objek tidak overlap. Untuk kasus segitiga vs kubus, terdapat 13 sumbu uji:

a. 3 sumbu utama kubus (X, Y, Z)
b. 1 sumbu normal segitiga
c. 9 sumbu hasil cross product edge segitiga dengan sumbu kubus

Langkah-langkah implementasi:

a. Ambil vertex segitiga f

```
bool Octree::intersect(OctreeNode* node, Face f) {

    Vertex v0 = initialVertices[f.a];

    Vertex v1 = initialVertices[f.b];

    Vertex v2 = initialVertices[f.c];

 //…
```

b. Tentukan titik tengah dan ukuran tengah kubus.

```
Vertex c{

    (node->min.x + node->max.x) * 0.5f,

    (node->min.y + node->max.y) * 0.5f,

    (node->min.z + node->max.z) * 0.5f

};

Vertex h{

    (node->max.x - node->min.x) * 0.5f,

    (node->max.y - node->min.y) * 0.5f,
```

```
    (node->max.z - node->min.z) * 0.5f

};
```

c. Translasi segitiga sehingga pusat voxel berada di (0,0,0) untuk mempermudah perhitungan SAT dan hitung edge untuk menghasilkan sumbu uji

```
Vertex tv0{v0.x - c.x, v0.y - c.y, v0.z - c.z};

Vertex tv1{v1.x - c.x, v1.y - c.y, v1.z - c.z};

Vertex tv2{v2.x - c.x, v2.y - c.y, v2.z - c.z};





Vertex e0{tv1.x - tv0.x, tv1.y - tv0.y, tv1.z - tv0.z};

Vertex e1{tv2.x - tv1.x, tv2.y - tv1.y, tv2.z - tv1.z};

Vertex e2{tv0.x - tv2.x, tv0.y - tv2.y, tv0.z - tv2.z};
```

d. Uji sumbu hasil cross product edge segitiga dengan sumbu kubus

Uji setiap sumbu dengan langkah-langkah:

i. Proyeksikan ketiga vertex segitiga
ii. Bandingkan dengan radius proyeksi box
iii. Jika tidak overlap, return false

```
auto axisTest = [&](float a, float b, float fa, float fb,
float v0a, float v0b, float v1a, float v1b, float v2a, float
v2b,float ha, float hb) {

    float p0 = a * v0a - b * v0b;

    float p1 = a * v1a - b * v1b;

    float p2 = a * v2a - b * v2b;
```

```
    float minP = std::min(p0, std::min(p1, p2));

    float maxP = std::max(p0, std::max(p1, p2));



    float rad = fa * ha + fb * hb;

    return !(minP > rad || maxP < -rad);

};



float fex = std::abs(e0.x);

float fey = std::abs(e0.y);

float fez = std::abs(e0.z);




if (!axisTest(e0.z, e0.y, fez, fey, tv0.y, tv0.z, tv1.y,
tv1.z, tv2.y, tv2.z, h.y, h.z)) return false;

if (!axisTest(e0.z, e0.x, fez, fex, tv0.x, tv0.z, tv1.x,
tv1.z, tv2.x, tv2.z, h.x, h.z)) return false;

if (!axisTest(e0.y, e0.x, fey, fex, tv0.x, tv0.y, tv1.x,
tv1.y, tv2.x, tv2.y, h.x, h.y)) return false;



fex = std::abs(e1.x);

fey = std::abs(e1.y);

fez = std::abs(e1.z);
```

```
if (!axisTest(e1.z, e1.y, fez, fey, tv0.y, tv0.z, tv1.y,
tv1.z, tv2.y, tv2.z, h.y, h.z)) return false;

if (!axisTest(e1.z, e1.x, fez, fex, tv0.x, tv0.z, tv1.x,
tv1.z, tv2.x, tv2.z, h.x, h.z)) return false;

if (!axisTest(e1.y, e1.x, fey, fex, tv0.x, tv0.y, tv1.x,
tv1.y, tv2.x, tv2.y, h.x, h.y)) return false;



fex = std::abs(e2.x);

fey = std::abs(e2.y);

fez = std::abs(e2.z);



if (!axisTest(e2.z, e2.y, fez, fey, tv0.y, tv0.z, tv1.y,
tv1.z, tv2.y, tv2.z, h.y, h.z)) return false;

if (!axisTest(e2.z, e2.x, fez, fex, tv0.x, tv0.z, tv1.x,
tv1.z, tv2.x, tv2.z, h.x, h.z)) return false;

if (!axisTest(e2.y, e2.x, fey, fex, tv0.x, tv0.y, tv1.x,
tv1.y, tv2.x, tv2.y, h.x, h.y)) return false;
```

e. Uji sumbu X, Y, Z

Mengecek apakah segitiga sepenuhnya di luar box pada sumbu x, y, z

```
auto min3 = [](float a, float b, float c) {

    return std::min(a, std::min(b, c));

};

auto max3 = [](float a, float b, float c) {

    return std::max(a, std::max(b, c));
```

```
};


if (max3(tv0.x, tv1.x, tv2.x) < -h.x || min3(tv0.x, tv1.x,
tv2.x) > h.x) return false;

if (max3(tv0.y, tv1.y, tv2.y) < -h.y || min3(tv0.y, tv1.y,
tv2.y) > h.y) return false;

if (max3(tv0.z, tv1.z, tv2.z) < -h.z || min3(tv0.z, tv1.z,
tv2.z) > h.z) return false;
```

f. Uji normal bidang segitiga

Menguji apakah box memotong bidang

```
Vertex normal{

    e0.y * e1.z - e0.z * e1.y,

    e0.z * e1.x - e0.x * e1.z,

    e0.x * e1.y - e0.y * e1.x

};


float d = -(normal.x * tv0.x + normal.y * tv0.y + normal.z *
tv0.z);


float r =

    h.x * std::abs(normal.x) +

    h.y * std::abs(normal.y) +

    h.z * std::abs(normal.z);
```

```
float s = normal.x * 0 + normal.y * 0 + normal.z * 0 + d;

if (std::abs(s) > r) return false;
```

g. Jika lolos semua uji, return true

```
return true;

}
```

3. Combine

Tahap ini dilakukan pada

```
void Octree::combine(const OctreeNode* node)
```

Tahap ini menggabungkan hasil dari semua leaf node menjadi voxel.

Langkah-Langkah:

a. Hitung statistik node:

```
nodeCount[node->depth]++;
```

b. Jika node adalah leaf dan berisi face, konversi node menjadi voxel:

```
if (node->isLeaf && !node->intersectFaces.empty()) {

            collectVoxels(node);

            return;

    }
```

Voxel dikumpulkan menggunakan bounding box node

```
void Octree::collectVoxels(const OctreeNode* node){
```

```
Vertex min = node->min;

Vertex max = node->max;

int startIdx = (int)voxelizedVertices.size();

// 8 Vertex kubus

voxelizedVertices.push_back({min.x, min.y, min.z}); // 0

voxelizedVertices.push_back({max.x, min.y, min.z}); // 1

voxelizedVertices.push_back({max.x, max.y, min.z}); // 2

voxelizedVertices.push_back({min.x, max.y, min.z}); // 3

voxelizedVertices.push_back({min.x, min.y, max.z}); // 4

voxelizedVertices.push_back({max.x, min.y, max.z}); // 5

voxelizedVertices.push_back({max.x, max.y, max.z}); // 6

voxelizedVertices.push_back({min.x, max.y, max.z}); // 7


// 12 sisi segitiga (6 sisi kubus)

//bottom

voxelizedFaces.push_back({startIdx + 0, startIdx + 1, startIdx + 2});

voxelizedFaces.push_back({startIdx + 0, startIdx + 2, startIdx + 3});

// top

voxelizedFaces.push_back({startIdx + 4, startIdx + 5, startIdx + 6});

voxelizedFaces.push_back({startIdx + 4, startIdx + 6, startIdx + 7});

// front

voxelizedFaces.push_back({startIdx + 0, startIdx + 1, startIdx + 5});

voxelizedFaces.push_back({startIdx + 0, startIdx + 5, startIdx + 4});

// right

voxelizedFaces.push_back({startIdx + 1, startIdx + 2, startIdx + 6});
```

```
    voxelizedFaces.push_back({startIdx + 1, startIdx + 6, startIdx + 5});

    // back

    voxelizedFaces.push_back({startIdx + 2, startIdx + 3, startIdx + 7});

    voxelizedFaces.push_back({startIdx + 2, startIdx + 7, startIdx + 6});

    // left

    voxelizedFaces.push_back({startIdx + 3, startIdx + 0, startIdx + 4});

    voxelizedFaces.push_back({startIdx + 3, startIdx + 4, startIdx + 7});

}
```

c. Rekursi ke child:

```
for (int i = 0; i < 8; i++) {

            if (node->children[i])

                combine(node->children[i]);

    }
```

C. Kompleksitas Algoritma

Karena rekursi divide dipanggil 8 kali untuk setiap node, dan setiap rekursi memanggil divide dan loop sebanyak faces hingga mencapai maxDepth, maka Kompleksitas waktunya adalah

$$O(8^D \cdot F)$$

D = Kedalaman maksimum
F = Jumlah Face

Hasil Pengujian

1. Cow.obj

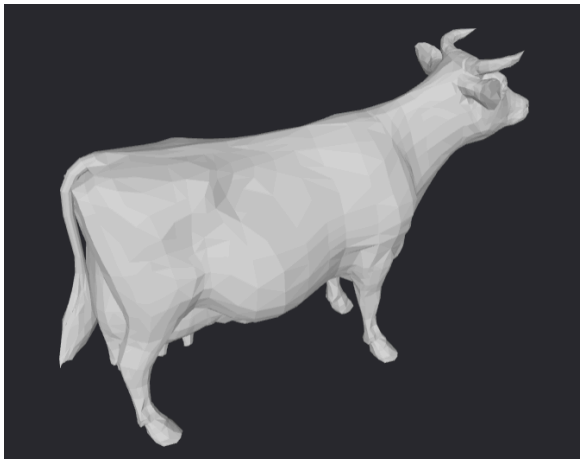
Input:

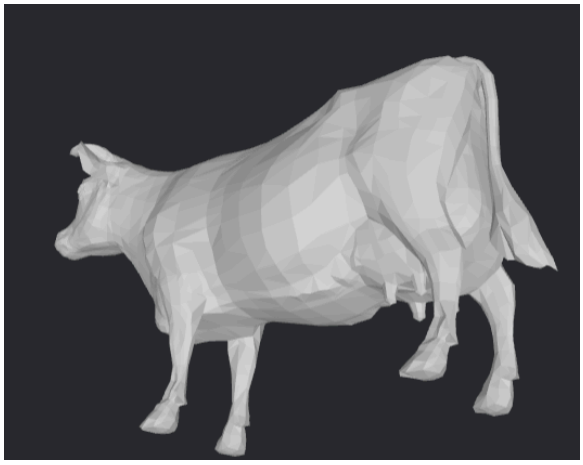
```
Masukkan path file .obj dari folder test: cow.obj
Masukkan kedalaman maksimum: 5
```

Output:

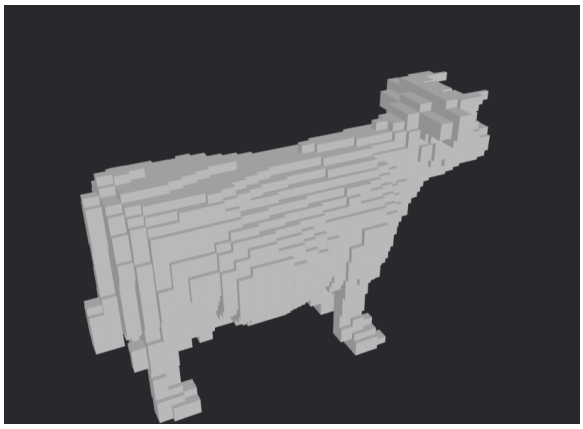
```
Banyaknya voxel: 3984
Banyaknya vertex: 31872
Banyaknya faces: 47808
0 : 1
1 : 8
2 : 64
3 : 384
4 : 1888
5 : 7616
2 : 16 (skipped)
3 : 148 (skipped)
4 : 936 (skipped)
5 : 3632 (skipped)
Kedalaman octree: 5
Lama waktu: 16 ms
Path file disimpan: test/voxelized_cow.obj
```

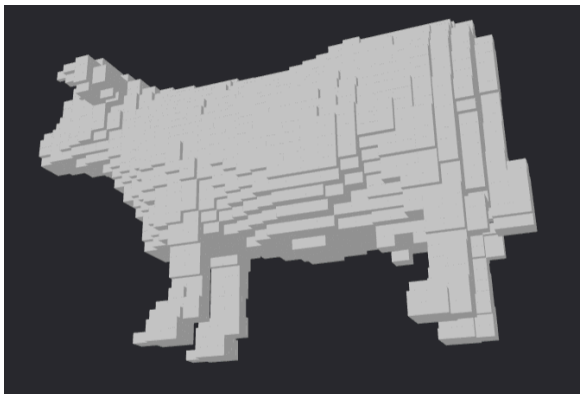
Objek 3D Awal:





Objek setelah di voxelisasi dengan maxDepth = 5:





2. Line.obj

   Input:

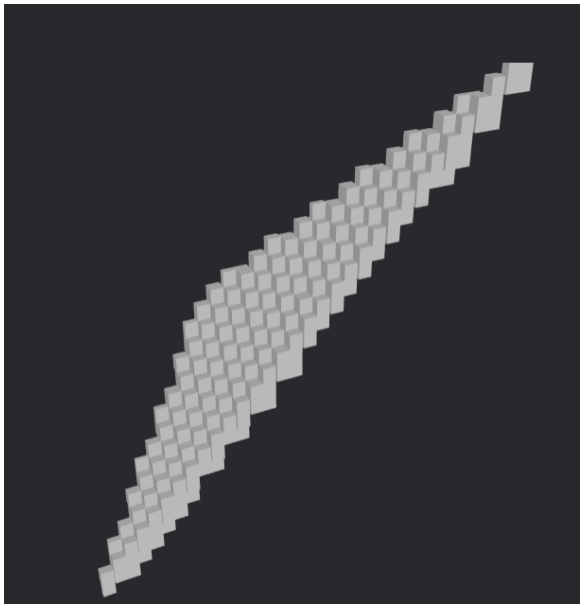
```
Masukkan path file .obj dari folder test: line.obj
Masukkan kedalaman maksimum: 5
```

   Output:

```
Banyaknya voxel: 930
Banyaknya vertex: 7440
Banyaknya faces: 11160
0 : 1
1 : 8
2 : 48
3 : 176
4 : 576
5 : 2040
1 : 2 (skipped)
2 : 26 (skipped)
3 : 104 (skipped)
4 : 321 (skipped)
5 : 1110 (skipped)
Kedalaman octree: 5
Lama waktu: 8 ms
Path file disimpan: test/voxelized_line.obj
```

Objek 3D Awal:

Objek setelah di voxelisasi dengan maxDepth = 5:



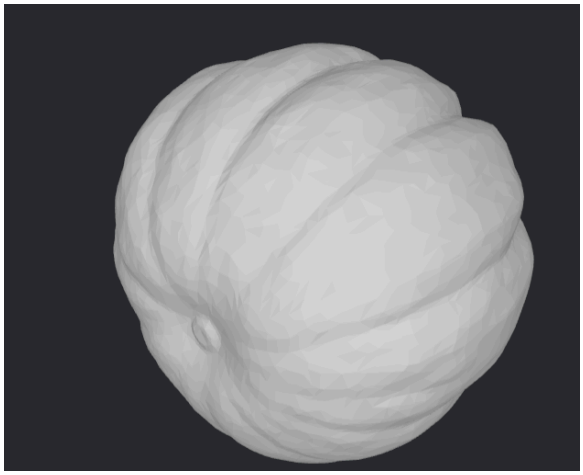
3. Pumpkin.obj
   Input:

```
Masukkan path file .obj dari folder test: pumpkin.obj
Masukkan kedalaman maksimum: 5
```

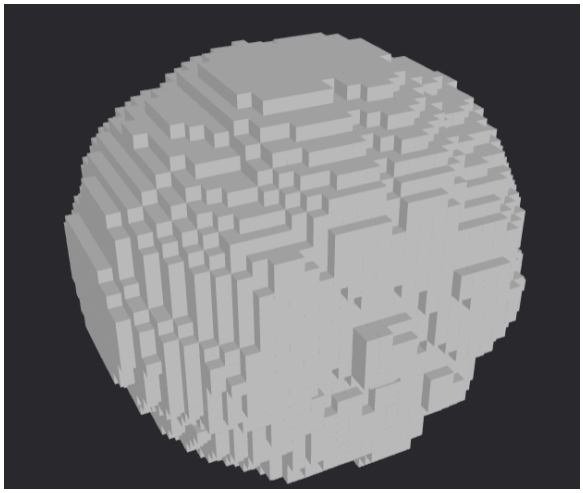
   Output:

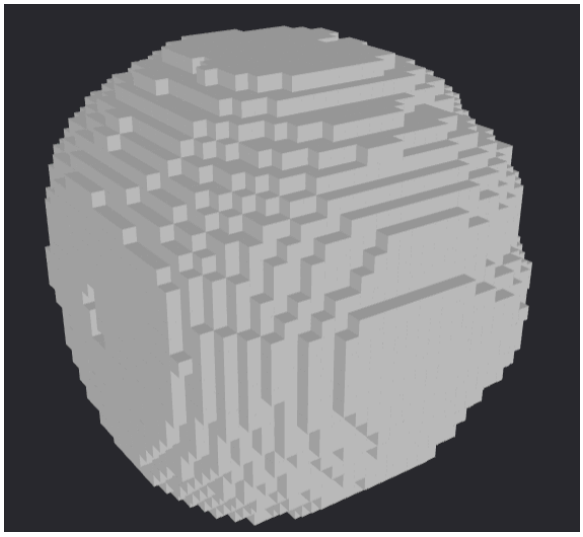
```
Banyaknya voxel: 5159
Banyaknya vertex: 41272
Banyaknya faces: 61908
0 : 1
1 : 8
2 : 64
3 : 448
4 : 2296
5 : 9912
2 : 8 (skipped)
3 : 161 (skipped)
4 : 1057 (skipped)
5 : 4753 (skipped)
Kedalaman octree: 5
Lama waktu: 32 ms
Path file disimpan: test/voxelized_pumpkin.obj
```

   Objek 3D Awal:



Objek setelah di voxelisasi dengan maxDepth = 5:





4. Teapot.obj
   Input:

```
Masukkan path file .obj dari folder test: teapot.obj
Masukkan kedalaman maksimum: 5
```

   Output:

```
Error di line 3: keyword tidak valid (g)
```

   Objek 3D Awal:

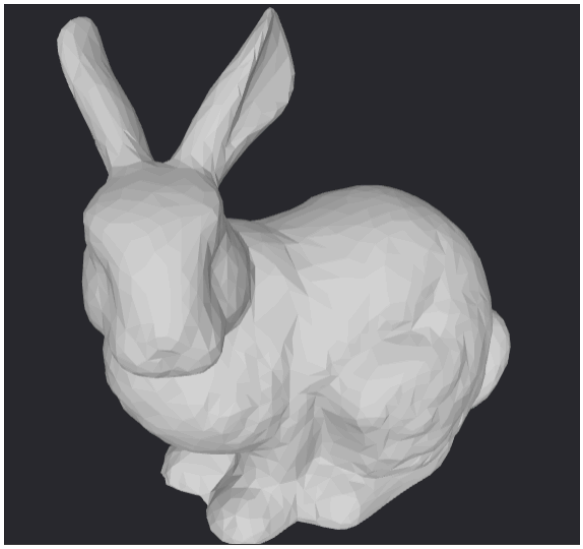
   Objek setelah di voxelisasi dengan maxDepth = 5:
   Program menolak input karena mengandung line dengan format tidak valid (bukan v x y z atau f i j k)
5. Bunny.obj
   Input:

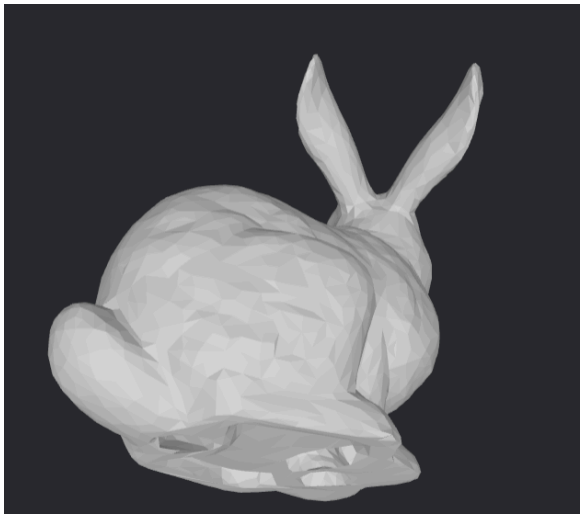
```
Masukkan path file .obj dari folder test: bunny.obj
Masukkan kedalaman maksimum: 5
```

Output:

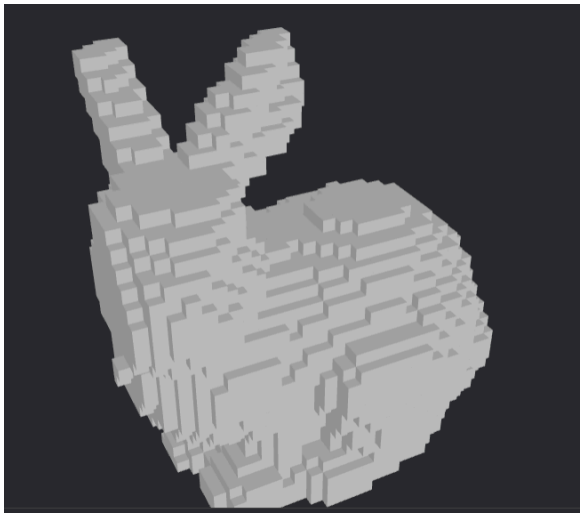
```
Banyaknya voxel: 4040
Banyaknya vertex: 32320
Banyaknya faces: 48480
0 : 1
1 : 8
2 : 64
3 : 376
4 : 1792
5 : 7744
2 : 17 (skipped)
3 : 152 (skipped)
4 : 824 (skipped)
5 : 3704 (skipped)
Kedalaman octree: 5
Lama waktu: 15 ms
Path file disimpan: test/voxelized_bunny.obj
```

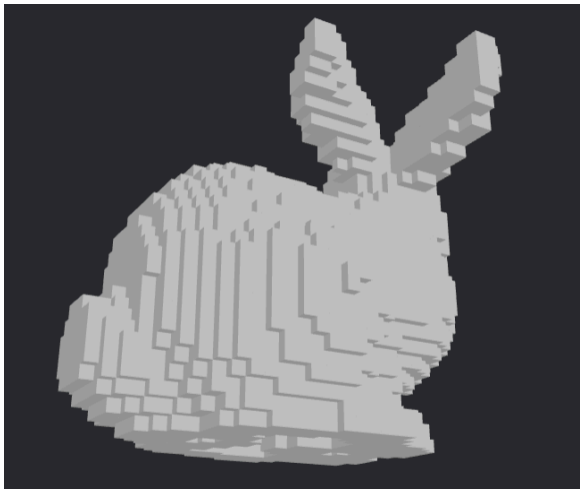
Objek 3D awal:





Objek setelah di voxelisasi dengan maxDepth = 5:





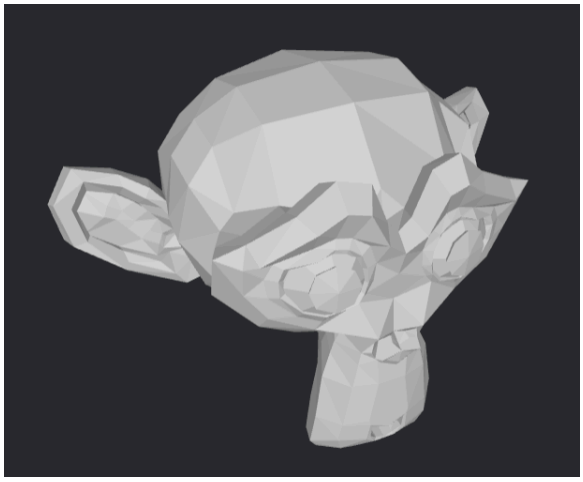
6. Monkey.obj
   Input:

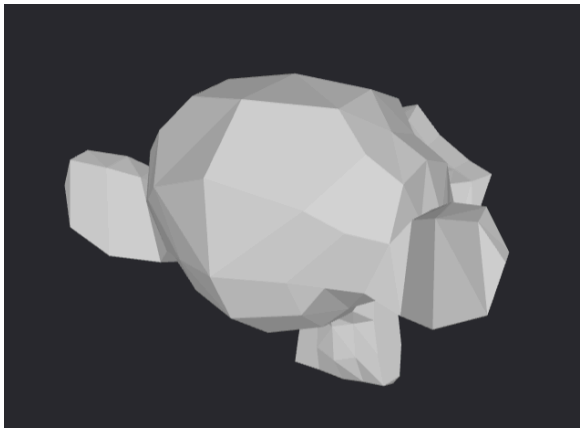
```
Masukkan path file .obj dari folder test: monkey.obj
Masukkan kedalaman maksimum: 5
```

   Output:

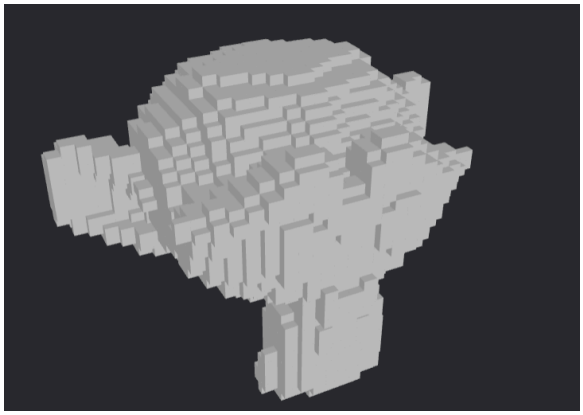
```
Banyaknya voxel: 4134
Banyaknya vertex: 33072
Banyaknya faces: 49608
0 : 1
1 : 8
2 : 64
3 : 400
4 : 1744
5 : 7456
2 : 14 (skipped)
3 : 182 (skipped)
4 : 812 (skipped)
5 : 3322 (skipped)
Kedalaman octree: 5
Lama waktu: 11 ms
Path file disimpan: test/voxelized_monkey.obj
```

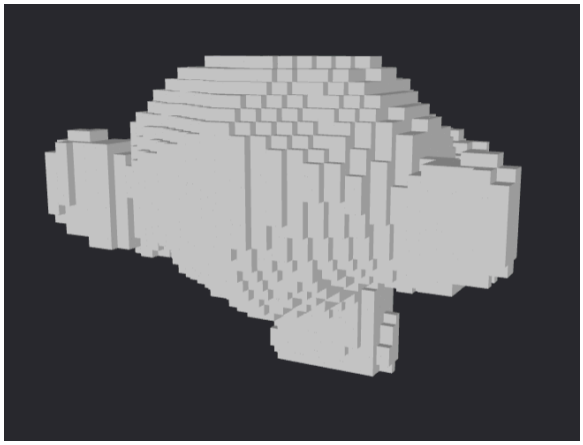
   Objek 3D awal:





Objek setelah di voxelisasi dengan maxDepth = 5:





Lampiran

Pranala repositori: https://github.com/wzlyy/Tucil2_13524054

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	
6	[Bonus] Program menggunakan *concurrency*	✓	
7	[Bonus] Program dapat menampilkan visualisasi .obj		✓

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Wildan Abdurrahman Ghazali